

Complex SQL Joins + Subqueries + All Major Combinations (MySQL)

The examples below use a simple business domain with:

- customers
- orders
- order_items
- products
- employees
- departments

The goal is to train freshers from beginner → advanced join understanding.

1. SAMPLE TABLES

```
-- =====  
-- CUSTOMERS  
-- =====
```

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  city VARCHAR(50)  
);
```

```
INSERT INTO customers VALUES  
(1, 'Arjun', 'Mumbai'),  
(2, 'Priya', 'Pune'),  
(3, 'Rahul', 'Delhi'),  
(4, 'Sneha', 'Chennai');
```

```
-- =====  
-- ORDERS  
-- =====
```

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  employee_id INT,  
  total_amount DECIMAL(10,2)  
);
```

```
INSERT INTO orders VALUES  
(101, 1, '2026-01-10', 1, 5000),  
(102, 1, '2026-01-15', 2, 7000),  
(103, 2, '2026-01-20', 1, 3000),  
(104, 3, '2026-02-01', 3, 10000);
```

```
-- =====  
-- PRODUCTS  
-- =====
```

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  price DECIMAL(10,2)  
);
```

```
INSERT INTO products VALUES  
(1, 'Laptop', 'Electronics', 60000),  
(2, 'Mobile', 'Electronics', 30000),  
(3, 'Shoes', 'Fashion', 4000),  
(4, 'Watch', 'Accessories', 5000);
```

```
-- =====  
-- ORDER ITEMS  
-- =====
```

```
CREATE TABLE order_items (  
  order_item_id INT PRIMARY KEY,  
  order_id INT,  
  product_id INT,  
  quantity INT
```

```
);
```

```
INSERT INTO order_items VALUES
```

```
(1, 101, 1, 1),
```

```
(2, 101, 4, 2),
```

```
(3, 102, 2, 1),
```

```
(4, 103, 3, 3),
```

```
(5, 104, 1, 1);
```

```
-- =====
```

```
-- DEPARTMENTS
```

```
-- =====
```

```
CREATE TABLE departments (
```

```
    department_id INT PRIMARY KEY,
```

```
    department_name VARCHAR(100)
```

```
);
```

```
INSERT INTO departments VALUES
```

```
(1, 'Sales'),
```

```
(2, 'IT'),
```

```
(3, 'HR');
```

```
-- =====
```

```
-- EMPLOYEES
```

```
-- =====
```

```
CREATE TABLE employees (
```

```
    employee_id INT PRIMARY KEY,
```

```
    employee_name VARCHAR(100),
```

```
    department_id INT,
```

```
    manager_id INT,
```

```
    salary DECIMAL(10,2)
```

```
);
```

```
INSERT INTO employees VALUES
```

```
(1, 'Karan', 1, NULL, 80000),
```

```
(2, 'Meena', 1, 1, 60000),
```

```
(3, 'Vikram', 2, 1, 90000),
```

```
(4, 'Anjali', 3, 3, 50000);
```

2. INNER JOIN

Returns matching records from both tables

-- Get customer orders

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_date,
  o.total_amount
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id;
```

3. LEFT JOIN

Returns all records from LEFT table + matching from RIGHT

-- Show all customers even if no orders exist

```
SELECT
  c.customer_name,
  o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id;
```

4. RIGHT JOIN

Returns all records from RIGHT table

-- Show all orders and matching customers

```
SELECT
  c.customer_name,
  o.order_id
FROM customers c
RIGHT JOIN orders o
  ON c.customer_id = o.customer_id;
```

5. CROSS JOIN

Produces cartesian product

-- Every customer with every product

```
SELECT
  c.customer_name,
  p.product_name
FROM customers c
CROSS JOIN products p;
```

6. SELF JOIN

Same table joined with itself

-- Employee and manager relationship

```
SELECT
  e.employee_name AS employee,
  m.employee_name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

7. MULTIPLE INNER JOINS

Joining many tables together

-- Customer orders with product details

```
SELECT
  c.customer_name,
  o.order_id,
  p.product_name,
  oi.quantity
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
INNER JOIN products p
  ON oi.product_id = p.product_id;
```

8. INNER + LEFT JOIN COMBINATION

-- Employees with department and orders handled

```
SELECT
  e.employee_name,
  d.department_name,
  o.order_id
FROM employees e
INNER JOIN departments d
  ON e.department_id = d.department_id
LEFT JOIN orders o
  ON e.employee_id = o.employee_id;
```

9. LEFT JOIN WITH WHERE

Important concept

-- Customers who never placed orders

```
SELECT
  c.customer_name
```

```
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

10. FULL OUTER JOIN (MYSQL WORKAROUND)

MySQL does not support FULL OUTER JOIN directly

-- FULL OUTER JOIN simulation

```
SELECT
  c.customer_name,
  o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
```

UNION

```
SELECT
  c.customer_name,
  o.order_id
FROM customers c
RIGHT JOIN orders o
  ON c.customer_id = o.customer_id;
```

11. SUBQUERY IN WHERE

-- Customers who placed orders above average amount

```
SELECT
  customer_name
FROM customers
WHERE customer_id IN (
  SELECT customer_id
  FROM orders
```

```
WHERE total_amount >
(
    SELECT AVG(total_amount)
    FROM orders
)
);
```

12. SUBQUERY WITH INNER JOIN

-- Get highest order per customer

```
SELECT
    c.customer_name,
    o.order_id,
    o.total_amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.total_amount =
(
    SELECT MAX(o2.total_amount)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
);
```

13. CORRELATED SUBQUERY + JOIN

Subquery depends on outer query

-- Employees earning above department average

```
SELECT
    e.employee_name,
    e.salary,
    d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id
WHERE e.salary >
```



```
(  
  SELECT AVG(e2.salary)  
  FROM employees e2  
  WHERE e2.department_id = e.department_id  
);
```

14. EXISTS WITH JOIN

-- Customers having at least one order

```
SELECT  
  c.customer_name  
FROM customers c  
WHERE EXISTS  
(  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
);
```

15. NOT EXISTS WITH JOIN

-- Customers without any orders

```
SELECT  
  c.customer_name  
FROM customers c  
WHERE NOT EXISTS  
(  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
);
```

16. JOIN WITH AGGREGATION

-- Total sales per customer

```
SELECT
  c.customer_name,
  SUM(o.total_amount) AS total_sales
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

17. JOIN + GROUP BY + HAVING

-- Customers with sales above 5000

```
SELECT
  c.customer_name,
  SUM(o.total_amount) AS total_sales
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING SUM(o.total_amount) > 5000;
```

18. DERIVED TABLE + JOIN

Subquery in FROM clause

-- Department average salary

```
SELECT
  d.department_name,
  x.avg_salary
FROM
  (
    SELECT
      department_id,
      AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
  ) x
INNER JOIN departments d
```

```
ON x.department_id = d.department_id;
```

19. JOIN WITH CASE

```
-- Order classification
```

```
SELECT
  c.customer_name,
  o.total_amount,
  CASE
    WHEN o.total_amount >= 8000 THEN 'HIGH'
    WHEN o.total_amount >= 5000 THEN 'MEDIUM'
    ELSE 'LOW'
  END AS order_category
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id;
```

20. MULTI LEVEL SUBQUERY + JOIN

```
-- Product ordered in highest quantity
```

```
SELECT
  p.product_name,
  oi.quantity
FROM products p
INNER JOIN order_items oi
  ON p.product_id = oi.product_id
WHERE oi.quantity =
(
  SELECT MAX(quantity)
  FROM order_items
);
```

21. JOIN WITH WINDOW FUNCTION

```
-- Rank customers by sales
```

```
SELECT
  c.customer_name,
  SUM(o.total_amount) AS total_sales,
  RANK() OVER (
    ORDER BY SUM(o.total_amount) DESC
  ) AS sales_rank
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

22. COMPLEX JOIN WITH MULTIPLE CONDITIONS

-- Electronics orders only

```
SELECT
  c.customer_name,
  p.product_name,
  p.category
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
INNER JOIN products p
  ON oi.product_id = p.product_id
  AND p.category = 'Electronics';
```

23. SUBQUERY INSIDE JOIN

-- Latest order per customer

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_date
FROM customers c
INNER JOIN
```

```
(
  SELECT
    customer_id,
    MAX(order_date) AS latest_order
  FROM orders
  GROUP BY customer_id
) x
ON c.customer_id = x.customer_id
INNER JOIN orders o
  ON o.customer_id = x.customer_id
  AND o.order_date = x.latest_order;
```

24. COMPLEX BUSINESS REPORT QUERY

-- Full business report

```
SELECT
  c.customer_name,
  o.order_id,
  p.product_name,
  oi.quantity,
  e.employee_name,
  d.department_name,
  o.total_amount
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN employees e
  ON o.employee_id = e.employee_id
INNER JOIN departments d
  ON e.department_id = d.department_id
INNER JOIN order_items oi
  ON o.order_id = oi.order_id
INNER JOIN products p
  ON oi.product_id = p.product_id;
```

IMPORTANT INTERVIEW POINTS

Concept

Important Point

INNER JOIN	Only matching rows
LEFT JOIN	All left rows
RIGHT JOIN	All right rows
CROSS JOIN	Cartesian product
SELF JOIN	Same table join
EXISTS	Checks existence
NOT EXISTS	Checks absence
Correlated Subquery	Runs for every outer row
Derived Table	Subquery inside FROM
HAVING	Filters grouped rows
WHERE	Filters before grouping
Window Functions	Ranking without collapsing rows

WHEN TO USE WHICH JOIN

Scenario	Join Type
Matching records only	INNER JOIN
All customers even without orders	LEFT JOIN
Hierarchical data	SELF JOIN
Product combinations	CROSS JOIN
Missing records analysis	LEFT JOIN + IS NULL
Summary reports	JOIN + GROUP BY
Top N per group	JOIN + Window Function
Latest record	JOIN + Subquery
Existence checking	EXISTS

1. AGGREGATE FUNCTIONS

What are Aggregate Functions?

Aggregate functions perform calculations on multiple rows and return a single result.

They are commonly used with:

- GROUP BY
 - HAVING
 - Reports
 - Analytics
 - Dashboards
-

Sample Table

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  emp_name VARCHAR(50),  
  department VARCHAR(50),  
  salary DECIMAL(10,2),  
  age INT,  
  city VARCHAR(50)  
);
```

```
INSERT INTO employees VALUES  
(1, 'Arun', 'IT', 75000, 28, 'Chennai'),  
(2, 'Priya', 'HR', 50000, 30, 'Bangalore'),  
(3, 'Rahul', 'IT', 90000, 35, 'Hyderabad'),  
(4, 'Sneha', 'Finance', 65000, 32, 'Mumbai'),  
(5, 'Vikram', 'IT', 80000, 29, 'Pune'),  
(6, 'Kiran', 'HR', 52000, 31, 'Delhi'),  
(7, 'Meena', 'Finance', 70000, 27, 'Chennai'),  
(8, 'Ravi', 'IT', 95000, 40, 'Bangalore');
```

2. COUNT()

Purpose

Counts rows.

COUNT(*)

```
-- Count total number of employees  
SELECT COUNT(*) AS total_employees  
FROM employees;
```

Output

8

COUNT(column)

```
-- Count employees having salary values  
SELECT COUNT(salary) AS salary_count  
FROM employees;
```

Note

- Ignores NULL values
-

COUNT(DISTINCT)

```
-- Count unique departments  
SELECT COUNT(DISTINCT department) AS unique_departments  
FROM employees;
```

3. SUM()

Purpose

Adds numeric values.


```
-- Calculate total salary of all employees
SELECT SUM(salary) AS total_salary
FROM employees;
```

SUM with GROUP BY

```
-- Total salary department wise
SELECT department,
       SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

4. AVG()

Purpose

Calculates average value.

```
-- Average employee salary
SELECT AVG(salary) AS average_salary
FROM employees;
```

AVG with GROUP BY

```
-- Average salary by department
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

5. MIN()

Purpose

Returns minimum value.

```
-- Minimum salary
SELECT MIN(salary) AS minimum_salary
FROM employees;
```

MIN with GROUP BY

```
-- Lowest salary in each department
SELECT department,
       MIN(salary) AS lowest_salary
FROM employees
GROUP BY department;
```

6. MAX()

Purpose

Returns maximum value.

```
-- Highest salary
SELECT MAX(salary) AS highest_salary
FROM employees;
```

MAX with GROUP BY

```
-- Highest salary department wise
SELECT department,
       MAX(salary) AS highest_salary
FROM employees
GROUP BY department;
```

7. GROUP_CONCAT()

Purpose

Combines multiple row values into one string.

```
-- Combine employee names department wise
SELECT department,
       GROUP_CONCAT(emp_name) AS employee_names
FROM employees
GROUP BY department;
```

8. GROUP BY

Purpose

Groups rows having same values.

```
-- Count employees in each department
SELECT department,
       COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

9. HAVING

Purpose

Filters grouped data.

```
-- Departments having more than 2 employees
SELECT department,
       COUNT(*) AS total_employees
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

10. AGGREGATE FUNCTIONS WITH WHERE

```
-- Average salary of IT employees only
```

```
SELECT AVG(salary) AS avg_it_salary
FROM employees
WHERE department = 'IT';
```

11. MULTIPLE AGGREGATE FUNCTIONS

```
-- Multiple aggregate calculations together
SELECT department,
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM employees
GROUP BY department;
```

12. WINDOW FUNCTIONS

What are Window Functions?

Window functions:

- Perform calculations across rows
- Do NOT collapse rows
- Return row-level details with calculations

Introduced in MySQL 8.0.

Syntax

```
FUNCTION() OVER (
    PARTITION BY column
    ORDER BY column
)
```

Difference Between Aggregate and Window Functions

Aggregate Function	Window Function
Groups rows	Keeps rows
Returns one row per group	Returns all rows
Used with GROUP BY	Used with OVER()
Summary output	Detailed analytics

13. ROW_NUMBER()

Purpose

Assigns unique row number.

```
-- Assign row number based on salary
SELECT emp_name,
       department,
       salary,
       ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_num
FROM employees;
```

ROW_NUMBER with PARTITION BY

```
-- Rank employees within each department
SELECT emp_name,
       department,
       salary,
       ROW_NUMBER() OVER(
         PARTITION BY department
         ORDER BY salary DESC
       ) AS department_rank
FROM employees;
```

14. RANK()

Purpose

Assigns ranking with gaps.

```
-- Salary ranking
SELECT emp_name,
       salary,
       RANK() OVER(ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Difference Between ROW_NUMBER and RANK

ROW_NUMBER

No duplicate rank

No gaps

RANK

Same values share same rank

Gaps occur

15. DENSE_RANK()

Purpose

Ranking without gaps.

```
-- Dense ranking by salary
SELECT emp_name,
       salary,
       DENSE_RANK() OVER(ORDER BY salary DESC) AS dense_rank
FROM employees;
```

Difference Between RANK and DENSE_RANK

RANK	DENSE_RANK
Gaps exist	No gaps
1,2,2,4	1,2,2,3

16. NTILE()

Purpose

Divides rows into groups.

d

17. LAG()

Purpose

Access previous row value.

```
-- Compare current salary with previous salary
SELECT emp_name,
       salary,
       LAG(salary) OVER(ORDER BY salary) AS previous_salary
FROM employees;
```

18. LEAD()

Purpose

Access next row value.

```
-- Compare current salary with next salary
SELECT emp_name,
       salary,
       LEAD(salary) OVER(ORDER BY salary) AS next_salary
FROM employees;
```

19. FIRST_VALUE()

Purpose

Returns first value in window.

```
-- First salary in ordered list
SELECT emp_name,
       salary,
       FIRST_VALUE(salary) OVER(
         ORDER BY salary DESC
       ) AS highest_salary
FROM employees;
```

20. LAST_VALUE()

Purpose

Returns last value in window.

```
-- Last salary value
SELECT emp_name,
       salary,
       LAST_VALUE(salary) OVER(
         ORDER BY salary
         ROWS BETWEEN UNBOUNDED PRECEDING
         AND UNBOUNDED FOLLOWING
       ) AS last_salary
FROM employees;
```

21. Running Total using SUM()

```
-- Running total of salary
SELECT emp_name,
       salary,
       SUM(salary) OVER(
         ORDER BY emp_id
       ) AS running_total
FROM employees;
```

22. Moving Average

```
-- Moving average of salaries
SELECT emp_name,
       salary,
       AVG(salary) OVER(
         ORDER BY emp_id
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_average
FROM employees;
```

23. PARTITION BY

Purpose

Divides data into logical groups.

```
-- Department wise average salary
SELECT emp_name,
       department,
       salary,
       AVG(salary) OVER(
         PARTITION BY department
       ) AS dept_avg_salary
FROM employees;
```

24. WINDOW FRAME CLAUSE

Purpose

Defines row range for calculation.

CURRENT ROW

```
-- Running salary total
SELECT emp_name,
       salary,
       SUM(salary) OVER(
         ORDER BY emp_id
         ROWS BETWEEN UNBOUNDED PRECEDING
         AND CURRENT ROW
       ) AS cumulative_salary
FROM employees;
```

PRECEDING

```
-- Include previous 2 rows
SELECT emp_name,
       salary,
       AVG(salary) OVER(
         ORDER BY emp_id
         ROWS BETWEEN 2 PRECEDING
         AND CURRENT ROW
       ) AS avg_salary
FROM employees;
```

FOLLOWING

```
-- Include next rows
SELECT emp_name,
       salary,
       SUM(salary) OVER(
         ORDER BY emp_id
         ROWS BETWEEN CURRENT ROW
```

```
        AND 2 FOLLOWING  
    ) AS future_sum  
FROM employees;
```

25. CUME_DIST()

Purpose

Calculates cumulative distribution.

```
-- Salary cumulative distribution  
SELECT emp_name,  
       salary,  
       CUME_DIST() OVER(  
         ORDER BY salary  
       ) AS cumulative_distribution  
FROM employees;
```

26. PERCENT_RANK()

Purpose

Returns percentile ranking.

```
-- Percent rank based on salary  
SELECT emp_name,  
       salary,  
       PERCENT_RANK() OVER(  
         ORDER BY salary  
       ) AS percent_rank  
FROM employees;
```

27. Complex Business Example

Find Top Paid Employee Per Department

```
SELECT *
FROM (
    SELECT emp_name,
           department,
           salary,
           ROW_NUMBER() OVER(
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rn
    FROM employees
) ranked_data
WHERE rn = 1;
```

28. Compare Employee Salary with Department Average

```
SELECT emp_name,
       department,
       salary,
       AVG(salary) OVER(
           PARTITION BY department
       ) AS department_average,

       salary -
       AVG(salary) OVER(
           PARTITION BY department
       ) AS salary_difference
FROM employees;
```

29. Find Salary Difference from Previous Employee

```
SELECT emp_name,
       salary,
       salary -
       LAG(salary) OVER(
```

```
ORDER BY salary
) AS salary_difference
FROM employees;
```

30. Aggregate vs Window Example

Aggregate Function

```
-- Returns one row per department
SELECT department,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Window Function

```
-- Returns all rows with department average
SELECT emp_name,
       department,
       salary,
       AVG(salary) OVER(
         PARTITION BY department
       ) AS dept_avg
FROM employees;
```

31. Interview Questions

Q1. Difference between GROUP BY and PARTITION BY?

GROUP BY	PARTITION BY
Reduces rows	Keeps rows
Aggregate functions	Window functions
One row per group	Detailed rows

Q2. Which MySQL version supports Window Functions?

MySQL 8.0+

Q3. Can Aggregate Functions be used as Window Functions?

Yes

Example:

```
SELECT emp_name,  
       salary,  
       SUM(salary) OVER() AS total_salary  
FROM employees;
```

32. Most Important Aggregate Functions

Function	Purpose
COUNT()	Count rows
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum
GROUP_CONCAT()	Combine values

33. Most Important Window Functions

Function	Purpose
ROW_NUMBER()	Unique numbering

RANK()	Ranking with gaps
DENSE_RANK()	Ranking without gaps
LAG()	Previous row
LEAD()	Next row
NTILE()	Divide groups
FIRST_VALUE()	First value
LAST_VALUE()	Last value

34. Real-Time Business Use Cases

Scenario	Function
Top N employees	ROW_NUMBER
Leaderboard ranking	RANK
Sales trend comparison	LAG
Forecast comparison	LEAD
Running totals	SUM OVER
Department analytics	PARTITION BY
Salary bands	NTILE

35. Important Notes

Aggregate Functions

- Used with **GROUP BY**
- Collapse rows
- Return summarized data

Window Functions

- Used with `OVER()`
 - Keep rows intact
 - Useful for analytics
-

36. Common Errors

Error 1

```
SELECT department, emp_name, AVG(salary)
FROM employees
GROUP BY department;
```

Problem

`emp_name` is not aggregated.

Correct Query

```
SELECT department,
       AVG(salary)
FROM employees
GROUP BY department;
```

37. Performance Tips

- Use indexes on:
 - `PARTITION BY` columns
 - `ORDER BY` columns
 - Avoid unnecessary window calculations.
 - Filter data before applying window functions.
-

38. Final Summary

Feature	Aggregate	Window
GROUP BY needed	Yes	No
OVER() needed	No	Yes
Row reduction	Yes	No
Analytics	Basic	Advanced
Ranking	No	Yes
Running totals	Limited	Excellent

38. SQL Scripts

```
/******WINDOWS FUNCTION COMPLETE GUIDE
ADVENTUREWORKS******/
/*
```

Function	Purpose
ROW_NUMBER()	Unique numbering
RANK()	Ranking with gaps
DENSE_RANK()	Ranking without gaps
LAG()	Previous row
LEAD()	Next row
NTILE()	Divide groups
FIRST_VALUE()	First value
LAST_VALUE()	Last value

```
*/
```

```
/*******/
```

```
/*
```

Real-Time Scenario	Business Use Cases Function
Top N employees	ROW_NUMBER
Leaderboard ranking	RANK
Sales trend comparison	LAG
Forecast comparison	LEAD

Running totals	SUM OVER
Department analytics	PARTITION BY
Salary bands	NTILE

```
/*
/*****
```

```
/*
```

ROW_NUMBER() is a SQL window function that assigns a unique sequential integer (starting from 1) to each row within a result set or partition.

Unlike ranking functions like RANK() or DENSE_RANK(), it does not skip numbers or assign identical values to ties;

every row receives a distinct number based on the specified order.

Syntax and Components

The function requires an OVER clause with an ORDER BY clause, which is mandatory to define the logical sequence for numbering:

```
ROW_NUMBER() OVER (
  [PARTITION BY column_list]
  ORDER BY sort_column [ASC | DESC]
)
```

PARTITION BY (Optional): Divides the result set into groups. The row counter resets to 1 for each new partition.

If omitted, the entire result set is treated as a single partition.

ORDER BY (Mandatory): Determines the order in which row numbers are assigned within each partition.

Common Use Cases

Pagination: Filtering results by row number range (e.g., WHERE row_num > 10 AND row_num <= 20) to display specific pages of data.

Top-N Queries: Identifying the top N records per group (e.g., highest salary per department) by filtering where row_num = 1.

Deduplication: Removing duplicate records by assigning numbers to duplicates and keeping only the row with row_num = 1.

```
*/
/*      STEP 1      */
CREATE TABLE sales (
  id SERIAL PRIMARY KEY,
  employee TEXT,
  department TEXT,
  sale_amount INT,
  sale_date DATE
);
```

/* STEP 2 */

INSERT INTO sales (employee, department, sale_amount, sale_date) VALUES

('Sean', 'Electronics', 300, '2023-01-01'),
('Anna', 'Electronics', 500, '2023-01-03'),
('Lubo', 'Electronics', 400, '2023-01-05'),
('David', 'Furniture', 700, '2023-01-02'),
('Eve', 'Furniture', 600, '2023-01-06'),
('Frank', 'Furniture', 750, '2023-01-07'),
('Grace', 'Clothing', 200, '2023-01-04'),
('Heidi', 'Clothing', 350, '2023-01-08'),
('Ivan', 'Clothing', 150, '2023-01-09'),
('Judy', 'Electronics', 550, '2023-01-10'),
('Karl', 'Furniture', 800, '2023-01-11'),
('Leo', 'Furniture', 720, '2023-01-12'),
('Mallory', 'Clothing', 280, '2023-01-13'),
('Nina', 'Clothing', 310, '2023-01-14'),
('Oscar', 'Electronics', 250, '2023-01-15'),
('Peggy', 'Furniture', 710, '2023-01-16'),
('Quinn', 'Furniture', 705, '2023-01-17'),
('Ruth', 'Clothing', 360, '2023-01-18'),
('Sam', 'Electronics', 460, '2023-01-19'),
('Trudy', 'Clothing', 400, '2023-01-20');

/* STEP 3 */

select * from sales;

/* STEP 4 */

SELECT

ROW_NUMBER() OVER (PARTITION BY department ORDER BY sale_amount DESC)
AS row_num,
employee,
department,
sale_amount
FROM sales;

/* STEP 5 */

SELECT

ROW_NUMBER() OVER (PARTITION BY department ORDER BY sale_amount DESC)
AS row_num,
RANK() OVER (PARTITION BY department ORDER BY sale_amount DESC) AS rank,

```
DENSE_RANK() OVER (PARTITION BY department ORDER BY sale_amount DESC) AS
dense_rank,
    employee,
    department,
    sale_amount
FROM sales;
```

```
/*      STEP 6      */
```

```
SELECT employee, sale_amount,
    NTILE(3) OVER (ORDER BY sale_amount) AS quartile
FROM sales;
```

```
/*      STEP 7      */
```

```
SELECT employee, sale_amount,
    LAG(sale_amount) OVER (ORDER BY sale_date) AS prev_sale,
    LEAD(sale_amount) OVER (ORDER BY sale_date) AS next_sale
FROM sales;
```

```
/*      STEP 8      */
```

```
SELECT employee, department, sale_amount,
    FIRST_VALUE(sale_amount) OVER (PARTITION BY department ORDER BY sale_date)
AS first_sale,
    LAST_VALUE(sale_amount) OVER (PARTITION BY department ORDER BY sale_date
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
    AS last_sale
FROM sales;
```